



**AI23331 — FUNDAMENTALS OF MACHINE LEARNING
MINI PROJECT REPORT**

ML-Based Productivity Tracker

A Desktop Application for Real-Time Activity Monitoring

Submitted by

**KASILINGAM M
(230701145)**

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**

**RAJALAKSHMI ENGINEERING COLLEGE
ANNA UNIVERSITY, CHENNAI**

MAY 2026



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project titled

"ML-Based Productivity Tracker: A Desktop Application for Real-Time Activity Monitoring and Machine Learning Classification" is the bonafide work of **"KASILINGAM M (230701145)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF GUIDE

Dr. _____

Associate Professor

Department of Computer Science and Engineering,
Rajalakshmi Engineering College, Chennai-602 105.

Submitted to Mini Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMNIER

ABSTRACT

The ML-Based Productivity Tracker is a cross-platform desktop application that monitors laptop usage in real-time, classifies each activity session using a trained XGBoost machine learning model, and presents productivity analytics through an interactive Tkinter dashboard. The system captures active window titles, application names, keystroke counts, and mouse interactions at two-second intervals, persisting all data to a local SQLite database. A synthetic dataset of over 6,000 activity sessions spanning 30 simulated days was generated across eight activity categories — Coding, Learning, Communication, Documents, Entertainment, Social Media, System, and Idle — and used to train the classifier.

The XGBoost model achieves an overall accuracy of 65.2% on the held-out test set. The most influential predictive feature is the activity category (importance: 39.6%), followed by idle-state detection (18.8%), keystroke count (10.1%), and mouse-click frequency (8.4%). A Bayesian prior-blending mechanism is employed for short or low-signal sessions to improve labelling reliability. The application is structured into four modules: Database, Activity Collector, ML Engine, and a Tkinter-based Dashboard. The dashboard provides four interactive tabs covering live monitoring, a daily report with pie and bar charts, a seven-day productivity trend, and detailed ML analysis including feature importance and per-class precision–recall metrics. The system is designed for personal productivity awareness and operates entirely offline without requiring any cloud infrastructure.

ACKNOWLEDGMENT

We thank the Almighty for guidance throughout this project. Our sincere gratitude to our Chairman Mr. S. Meganathan, B.E, F.I.E., Vice Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., and Chairperson Dr. (Mrs.) Thangam Meganathan, Ph.D., for providing excellent infrastructure and educational environment.

We express sincere thanks to Dr. S.N. Murugesan, M.E., Ph.D., our beloved Principal, and Dr. E. M. Malathy, M.E., Ph.D., Professor and Head of the Department of Computer Science and Engineering, for their constant support and encouragement.

We convey our deepest gratitude to our internal guide for invaluable guidance, technical feedback, and motivation throughout the project lifecycle.

KASILINGAM M 230701145

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGMENT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
1.1	GENERAL	1
1.2	OBJECTIVES	2
1.3	EXISTING SYSTEM	2
2	LITERATURE SURVEY	4
3	PROPOSED SYSTEM	8
3.1	SYSTEM ARCHITECTURE	8
3.2	HARDWARE & SOFTWARE REQUIREMENTS	9
3.3	DATASET DESCRIPTION	10
4	MODULE DESCRIPTION	12
4.1	DATABASE MODULE	12
4.2	ACTIVITY COLLECTOR MODULE	13
4.3	ML ENGINE MODULE	14
4.4	DASHBOARD MODULE	15
5	IMPLEMENTATION AND RESULTS	18
6	CONCLUSION AND FUTURE ENHANCEMENT	24
	REFERENCES	26

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	HARDWARE REQUIREMENTS	9
3.2	SOFTWARE REQUIREMENTS	10
3.3	ACTIVITY CATEGORIES AND LABELS	11
5.1	MODEL PERFORMANCE METRICS PER CLASS	20
5.2	FEATURE IMPORTANCE SCORES	21
5.3	CATEGORY PRIOR PROBABILITIES (SELECTED)	22

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE DIAGRAM	8
3.2	DATA FLOW DIAGRAM	9
4.1	DATABASE ENTITY-RELATIONSHIP DIAGRAM	13
4.2	ML ENGINE TRAINING PIPELINE	15
5.1	DASHBOARD — LIVE MONITOR TAB	19
5.2	DASHBOARD — TODAY'S REPORT TAB	20
5.3	FEATURE IMPORTANCE BAR CHART	21
5.4	PER-CLASS PRECISION AND RECALL CHART	22

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Expansion
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSV	Comma-Separated Values
4	DB	Database
5	GUI	Graphical User Interface
6	JSON	JavaScript Object Notation
7	ML	Machine Learning
8	OS	Operating System
9	pkl	Pickle file (serialised Python object)
10	SQL	Structured Query Language
11	SQLite	Self-contained Relational Database Engine
12	UI	User Interface
13	URL	Uniform Resource Locator
14	XGBoost	Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 GENERAL

In the era of digital work environments, individuals spend a significant proportion of their working hours interacting with computers — switching between coding environments, communication tools, entertainment platforms, and social media. While this versatility is a hallmark of modern productivity, it also presents a significant challenge: the inability to accurately measure how time is being allocated between genuinely productive and unproductive activities.

Traditional productivity tracking solutions — such as time-logging applications and browser extensions — rely primarily on manual input or simple rule-based URL filters, which are both effort-intensive and prone to misclassification. The emergence of machine learning provides an opportunity to automate this classification problem by learning patterns from historical activity data and generating reliable, real-time productivity scores.

This project presents the ML-Based Productivity Tracker, a single-file Python desktop application that captures real-time laptop activity (active window, application name, keystrokes, and mouse interactions), stores sessions in a SQLite database, trains an XGBoost classifier on labelled activity data, and surfaces productivity analytics through a four-tab Tkinter dashboard. The system is entirely self-contained, offline, and privacy-preserving.

1.2 OBJECTIVES

- To design and implement a real-time activity monitoring system for desktop environments across Windows, macOS, and Linux.
- To generate a realistic synthetic dataset of laptop activity sessions using probabilistic, time-of-day-biased simulation.
- To train a multi-class XGBoost classification model that predicts session labels (Productive, Neutral, Unproductive) from behavioural features.
- To build an interactive Tkinter dashboard with live monitoring, daily reports, weekly trend charts, and ML analysis tabs.
- To implement a Bayesian prior-blending mechanism to handle short or low-signal activity sessions more accurately.
- To store all activity data locally in SQLite, preserving user privacy with no dependency on cloud services.

1.3 EXISTING SYSTEM

Several commercial and open-source productivity tracking tools exist, each with notable limitations:

RescueTime is a widely-used time-tracking service that categorises applications and websites automatically. However, it requires cloud data upload, raising privacy concerns, and uses static rule-based classification with no machine learning component. It does not incorporate behavioural signals such as keystrokes or mouse activity.

Toggl Track and Clockify are time-logging tools that require manual entry to start and stop timers, making them impractical for passive, continuous tracking. Their accuracy depends entirely on user discipline.

ActivityWatch is an open-source self-hosted tracker that logs window focus and idle time. While privacy-preserving, it offers only basic rule-based categorisation and has no machine learning-based productivity scoring.

The proposed system addresses these gaps by combining passive real-time tracking, a trained XGBoost classifier with feature-engineered behavioural signals, Bayesian label blending for robustness, and fully local operation without any cloud dependency.

CHAPTER 2

LITERATURE SURVEY

A review of related work in the areas of activity recognition, productivity modelling, and machine learning for behavioural classification informs the design of this project.

2.1 Activity Recognition on Desktop Systems

Kapoor and Horvitz (2008) demonstrated that passive sensing of user behaviour through keyboard and mouse dynamics can reliably predict cognitive states and task interruption risk. Their work highlighted the importance of sub-session features — particularly keystroke rate and inter-click intervals — as proxies for engagement depth, a finding directly reflected in our feature set.

Czerwinski et al. (2004) conducted empirical studies on task fragmentation in knowledge workers, documenting that the average worker switches tasks every three minutes and takes

up to 23 minutes to regain full focus. This motivates the session-level granularity used in the present system.

2.2 Machine Learning for Behavioural Classification

Chen and Guestrin (2016) introduced XGBoost, the gradient boosted tree algorithm used in this project. XGBoost consistently ranks among the top performers in tabular data classification benchmarks, offering built-in regularisation, parallel tree construction, and handling of missing values — all practical advantages for the activity classification task.

Lundberg and Lee (2017) developed SHAP (SHapley Additive exPlanations), which provides model-agnostic feature importance explanations. Their approach underpins our use of XGBoost's native feature importance to identify the category encoding, idle state, and keystroke count as the most predictive signals.

2.3 Productivity and Time-Use Modelling

Mark et al. (2016) studied the relationship between digital distraction and self-reported productivity, finding that social media use and entertainment browsing correlated strongly with reduced task completion rates. This empirical evidence validates the Unproductive label assigned to social media and entertainment categories in our classification scheme.

Bailey and Konstan (2006) analysed the cost of interrupted focus in software developers, demonstrating that each task switch incurs a measurable performance penalty. This supports the use of continuous, passive monitoring rather than periodic user-initiated reporting.

2.4 Synthetic Data Generation for Behavioural Systems

The generation of synthetic behavioural data is an established technique in privacy-sensitive domains. Jordon et al. (2019) showed that synthetic tabular datasets produced by probabilistic simulation can achieve distributions closely matching real data while preserving privacy. Our data generator uses log-normal duration distributions, time-of-day productivity biases, and weekday probability weights to produce a realistic 6,000-session training corpus.

2.5 Bayesian Approaches to Uncertain Classification

Murphy (2012) provides a thorough treatment of naive Bayes and prior-blending in probabilistic classification. The category-prior blending used in our ML Engine for short sessions (below 180 seconds) follows this paradigm: when the model has insufficient signal from a brief interaction, the posterior is regularised toward the observed category prior, reducing erratic label assignments.

CHAPTER 3

PROPOSED SYSTEM

3.1 SYSTEM ARCHITECTURE

The system is architected as a multi-threaded desktop application comprising four loosely-coupled modules that communicate through a shared thread-safe event queue. The architecture is deliberately single-file to simplify deployment, with all state persisted in a local SQLite database.

The high-level data flow is as follows: the Activity Collector daemon thread polls the operating system every two seconds to retrieve the currently focused application and window title. Each detected application switch triggers a session record to be pushed onto the event queue and buffered for database insertion. Concurrently, the ML Engine daemon thread operates on a 30-second scoring cycle, retrieving all unscored sessions from the database, running inference through the trained XGBoost model, and posting the aggregated productivity score back onto the queue. The Tkinter Dashboard runs on the main thread, consuming queue events and refreshing its four tabs every 15 seconds.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

3.2.1 HARDWARE REQUIREMENTS

Component	Minimum Specification
Processor	Intel Core i3 (7th Gen) or equivalent, 1.6 GHz
RAM	4 GB DDR4
Storage	500 MB free disk space (for database and model)
Display	1280 x 720 minimum resolution
Operating System	Windows 10 / macOS 11 / Ubuntu 20.04 or later

3.2.2 SOFTWARE REQUIREMENTS

Software / Library	Version / Purpose
Python	3.9 or higher — core runtime
Tkinter	Built-in — GUI framework
XGBoost	1.7+ — classification model
scikit-learn	1.2+ — preprocessing and evaluation
pandas	1.5+ — data manipulation
NumPy	1.23+ — numerical operations

Matplotlib	3.6+ — chart rendering via TkAgg backend
joblib	1.2+ — model serialisation
SQLite3	Built-in — local database
pynput (optional)	Keystroke and mouse-click tracking

3.3 DATASET DESCRIPTION

A synthetic dataset was generated using a probabilistic simulation script (`data.py`) that models realistic laptop usage over 30 days. The generator produces sessions drawn from 33 distinct application profiles covering eight categories: Coding, Learning, Communication, Documents, Entertainment, Social Media, System, and Idle.

Each session record contains: timestamp, date, hour, day of week, application name, window title, URL (if browser-based), category, productivity label (Productive / Neutral / Unproductive), duration in seconds, idle flag, keystroke count, and mouse-click count. Session durations are sampled from a log-normal distribution centred on the category average duration, clamped between 10 seconds and 2 hours. Keyboard and mouse activity are sampled according to session length and idle state.

A time-of-day productivity bias is applied: morning hours (08:00–11:00) have a 65% boost to productive activity weights, while noon (12:00–13:00) increases distraction weights by 40%, reflecting documented patterns of knowledge worker focus cycles. Weekend sessions are skipped with 55% probability to model realistic usage patterns.

Category	Label	Avg Duration (s)	Example Apps
Coding	Productive	300–900	VS Code, PyCharm, Terminal
Learning	Productive	180–1200	Coursera, Stack Overflow, GitHub
Communication	Neutral	90–1800	Slack, Gmail, Zoom
Documents	Neutral	180–480	Google Docs, MS Word
Entertainment	Unproductive	600–5400	YouTube, Netflix, VLC
Social Media	Unproductive	180–360	Twitter, Reddit, Instagram
System	Neutral	60–90	File Explorer, Settings
Idle	Neutral	600	Screen Lock / Idle

Table 3.3: Activity Categories and Labels

CHAPTER 4

MODULE DESCRIPTION

4.1 DATABASE MODULE

The Database class manages all persistent storage using Python's built-in SQLite3 library. A connection-per-call pattern is used to ensure thread safety across the Activity Collector and ML Engine background threads. On initialisation, two tables are created if they do not already exist: sessions and predictions.

The sessions table stores one row per detected activity session and contains fifteen columns: id (auto-increment primary key), timestamp, date, hour, day_of_week, app_name, window_title, url, category, label, duration_sec, is_idle, keystrokes, mouse_clicks, and score (ML-assigned, nullable). An index on the date column accelerates the frequent date-filtered queries used by the dashboard.

The predictions table logs each ML scoring cycle's aggregate result (timestamp, score, label), providing a lightweight audit trail for debugging model behaviour over time. Key methods include insert_session, insert_many (bulk buffered inserts), update_scores (used by the ML Engine after inference), fetch_today, fetch_last_n_days, fetch_all, fetch_recent, and log_prediction.

4.2 ACTIVITY COLLECTOR MODULE

The ActivityCollector class runs as a daemon thread, polling the operating system every two seconds using platform-specific APIs. On Windows, it uses the Win32 ctypes interface (GetForegroundWindow, GetWindowTextW, GetModuleFileNameExW). On macOS, it invokes AppleScript via subprocess. On Linux, it uses the xdotool command-line utility.

Each poll detects application switches by comparing the current application name to the previously tracked application. On a switch event, the elapsed duration is calculated, and the session is classified using a two-tier rule engine: URL-based classification (most specific, covering 25 known domains) takes precedence, followed by application-name keyword matching across eight categories. Window title keyword fallback handles unmapped applications.

Keystroke and mouse-click counts are accumulated via pynput listeners (if installed) and reset to zero on each session boundary. Sessions are buffered in memory and flushed to the database every five seconds to reduce write frequency. A 5% probability random break is introduced per session to simulate realistic short interruptions such as coffee breaks.

4.3 ML ENGINE MODULE

The MLEngine class implements the full machine learning pipeline: feature engineering, model training, serialisation, loading, live session scoring, and daily re-scoring.

4.3.1 Feature Engineering

Eight features are constructed for each session record. Categorical features (category, day_of_week) are integer-encoded using pre-fitted scikit-learn LabelEncoders. A session_idx ordinal feature encodes the sequential position of each session within the dataset. Numeric features (duration_sec, keystrokes, mouse_clicks) are clipped and null-filled. The full feature set is:

Feature	Description
hour	Hour of day (0–23)
duration_sec	Session duration in seconds (clipped to 7,200)
is_idle	Binary flag: 1 if session is classified as Idle
keystrokes	Total keystrokes recorded during session
mouse_clicks	Total mouse clicks recorded during session
category_enc	Integer-encoded activity category (0–8)
day_enc	Integer-encoded day of week (0–6)
session_idx	Ordinal position within the training dataset

4.3.2 Model Training

An XGBoost multi-class classifier is trained with 150 estimators, maximum depth of 5, and learning rate of 0.1. The label encoder maps the three target classes (Productive, Neutral, Unproductive) to integers. A stratified 80/20 train-test split is applied. The trained model and label encoder are serialised using joblib and written to model.pkl and label_encoder.pkl respectively. Model metadata including accuracy, per-class metrics, feature importances, and category priors are written to model_meta.json.

4.3.3 Bayesian Prior Blending

For sessions with insufficient signal (duration below 180 seconds, zero keystrokes, or zero mouse clicks), the raw model probability output is blended with the observed category-level

prior distribution. The model weight ranges from 0.20 (for sessions under 30 seconds) to 0.55 (for sessions between 120 and 180 seconds). This regularisation prevents the model from assigning extreme labels to brief or passive sessions where behavioural signals are uninformative.

4.3.4 Productivity Score

The final productivity score (0–100) is computed as a linear combination of blended class probabilities: $\text{Score} = P(\text{Productive}) \times 100 + P(\text{Neutral}) \times 50$. Scores above 65 are displayed in green, between 35 and 65 in amber, and below 35 in red on the dashboard.

4.4 DASHBOARD MODULE

The `ProductivityApp` class extends `tk.Tk` and implements a four-tab notebook interface rendered with the clam Tkinter theme. All background work is communicated to the UI thread through a shared `queue.Queue`; the dashboard polls this queue every 500 milliseconds.

4.4.1 Live Monitor Tab

Displays a large productivity score card with a colour-coded progress bar, the currently active application and window title, and four aggregate statistics for the current day: Productive %, Focus hours, Distracted minutes, and Sessions today. A scrollable activity feed shows the 20 most recent sessions with timestamp, app, category, label, and ML score.

4.4.2 Today's Report Tab

Renders three Matplotlib charts embedded in the Tkinter canvas using the TkAgg backend: (1) a pie chart showing the proportion of time spent in each label class, (2) a horizontal bar chart showing time by category in hours, and (3) a line chart of average hourly productivity score throughout the day.

4.4.3 Weekly Charts Tab

Renders two charts: (1) a colour-coded bar chart of average daily productivity score over the past seven days, with reference lines at score thresholds 35 and 65, and (2) a stacked bar chart showing productive, neutral, and unproductive minutes per day.

4.4.4 ML Analysis Tab

Displays the model accuracy and training sample count. Two charts are rendered: (1) a horizontal bar chart of feature importance scores from the XGBoost model, and (2) a grouped bar chart of per-class precision and recall for the three label categories.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 IMPLEMENTATION OVERVIEW

The complete application is implemented in a single Python file (app.py) with approximately 900 lines of code. The synthetic data generator (data.py) produces the training corpus as a CSV file (activity_logs.csv). On first launch, the ML Engine checks for a saved model (model.pkl); if none is found, it trains a new model on the CSV data. The SQLite database (activity.db) is created automatically on startup.

To run the application, the user installs the required Python packages and executes: `python app.py`. On Windows, the active window tracking requires no additional dependencies. On Linux, `xdotool` must be installed separately. Keystroke and mouse tracking require `pynput` but degrade gracefully if absent.

5.2 MODEL PERFORMANCE

The XGBoost classifier was evaluated on a held-out test set of 1,200 sessions (20% of the 6,000-session dataset). The overall accuracy is 65.2%. Per-class performance metrics are presented below:

Class	Precision	Recall	F1-Score	Support
Neutral	0.609	0.734	0.666	548
Productive	0.692	0.644	0.667	478
Unproductive	0.758	0.414	0.535	174
Weighted Avg	0.664	0.652	0.647	1200

Table 5.1: Model Performance Metrics Per Class

The Neutral class achieves the highest recall (73.4%), reflecting the largest support and the relative simplicity of neutral classification boundaries. The Unproductive class has the highest precision (75.8%) but low recall (41.4%), indicating that when the model predicts unproductive, it is usually correct, but it misses approximately 59% of unproductive sessions. This trade-off is partly attributable to the class imbalance: Unproductive sessions constitute only 14.5% of the test set. The Bayesian prior-blending mechanism partially addresses this by increasing unproductive probability estimates for known high-distraction categories such as Social Media.

5.3 FEATURE IMPORTANCE ANALYSIS

Feature	Importance Score	Rank
category_enc	0.3957	1
is_idle	0.1875	2
keystrokes	0.1008	3
mouse_clicks	0.0841	4
duration_sec	0.0688	5
hour	0.0608	6
day_enc	0.0531	7
session_idx	0.0492	8

Table 5.2: Feature Importance Scores

The activity category encoding dominates with an importance of 39.6%, confirming that the type of application or website visited is by far the strongest predictor of productivity. The idle state flag contributes 18.8%, capturing the substantial signal of screen inactivity. Keystroke count (10.1%) and mouse clicks (8.4%) together account for nearly one-fifth of model weight, validating the inclusion of input hardware monitoring. Temporal features (hour, day) contribute modestly at 6.1% and 5.3% respectively.

5.4 CATEGORY PRIOR DISTRIBUTIONS

The category prior probabilities computed from the training data inform the Bayesian blending:

Category	P(Productive)	P(Neutral)	P(Unproductive)
Coding	0.663	0.311	0.026
Learning	0.634	0.354	0.012
Social Media	0.031	0.388	0.581
Entertainment	0.211	0.607	0.181
Idle	0.004	0.049	0.947
Communication	0.222	0.669	0.109

Table 5.3: Category Prior Probabilities (Selected)

These priors reveal strong category-label correlations: Coding and Learning sessions are predominantly Productive (>63%), Social Media sessions are predominantly Unproductive

(58%), and Idle sessions are almost entirely Unproductive (94.7%). Communication and Entertainment show mixed distributions, correctly classified as Neutral-leaning with moderate unproductive probability.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project presents a complete end-to-end system for real-time desktop productivity monitoring using machine learning. The ML-Based Productivity Tracker successfully integrates passive activity collection, a trained XGBoost multi-class classifier, Bayesian prior blending, and an interactive Tkinter dashboard into a single, offline, privacy-preserving Python application.

The XGBoost model achieves 65.2% accuracy on the synthetic test set, with category encoding identified as the dominant predictive feature. The Bayesian blending mechanism improves label stability for short, low-signal sessions. The dashboard provides actionable productivity analytics including live scoring, daily breakdowns by category and label, seven-day trend visualisation, and model diagnostics.

The system demonstrates that machine learning can meaningfully automate the classification of computer activity sessions without requiring cloud infrastructure or manual user input, providing a viable privacy-preserving alternative to commercial productivity trackers.

6.2 FUTURE ENHANCEMENTS

- **Deep Learning Integration:** Replace XGBoost with an LSTM or Transformer-based sequence model to capture temporal dependencies between consecutive application switches, potentially improving classification of interleaved productive and distracting sessions.
- **Browser Extension Integration:** Develop a companion browser extension to capture precise URL and page-content metadata, enabling finer-grained classification of browser activity (e.g., distinguishing tutorial reading from passive scrolling on the same domain).
- **Personalised Model Fine-Tuning:** Allow the system to retrain on the user's actual labelled data over time, gradually adapting the productivity classification to individual work patterns and preferences.
- **Pomodoro and Focus Session Integration:** Incorporate voluntary focus session timers to provide labelled ground-truth data that can improve training dataset quality for the personal fine-tuning pipeline.

- Natural Language Processing for Window Titles: Apply NLP-based entity extraction to window titles to infer task context (e.g., identifying a project name in a code editor title), enabling richer feature engineering.
- Notification and Intervention System: Generate non-intrusive desktop notifications when sustained unproductive activity is detected, nudging the user back to focused work without interrupting productive flow.
- Mobile Companion Application: Extend the tracker to Android and iOS to provide cross-device productivity scoring, including phone usage correlation with desktop activity.
- Cross-Platform Package Distribution: Package the application as a standalone executable using PyInstaller for Windows, macOS, and Linux, eliminating the need for manual Python environment setup.

REFERENCES

1. Bailey, B. P. and Konstan, J. A. (2006). On the need for attention-aware systems: Measuring the effects of interruption on task performance, error rate, and affective state. *Computers in Human Behavior*, 22(4), 685-708.
2. Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
3. Czerwinski, M., Horvitz, E. and Wilhite, S. (2004). A diary study of task switching and interruptions. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, 175-182.
4. Jordon, J., Yoon, J. and van der Schaar, M. (2019). PATE-GAN: Generating synthetic data with differential privacy guarantees. *International Conference on Learning Representations (ICLR)*.
5. Kapoor, A. and Horvitz, E. (2008). Experience sampling for building predictive user models: A comparative study. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 657-666.
6. Lundberg, S. M. and Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765-4774.
7. Mark, G., Iqbal, S., Czerwinski, M., Johns, P. and Sano, A. (2016). Neurotics can't focus: An in-situ study of online multitasking in the workplace. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1739-1744.
8. Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA.
9. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.
10. Scikit-learn: Machine Learning in Python. Pedregosa et al. (2011). *Journal of Machine Learning Research*, 12, 2825-2830.
11. SQLite Documentation. (2024). SQLite: A Self-Contained, Serverless, Zero-Configuration, Transactional SQL Database Engine. <https://www.sqlite.org/docs.html>
12. Tkinter Documentation. Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. <https://docs.python.org/3/library/tkinter.html>